AMRUTA INSTITUTE OF ENGINEERING AND MANAGEMENT SCIENCES – POLYTECHNIC

Course: Database System Concepts and PL/SQL

Course Code: 20CS34P

Week 1

OVERVIEW OF DB

Data: Data is a meaningful information, it can be in the form of video, audio, text, image, numbers or any Combination of these.

(or)

Data is facts that can be recorded.

Database: It is a collection of related data, having implicit meaning.

It is designed to meet the requirements of an organisation.

Ex: Student database Containing students register number, name, DoB, marks etc)

Commercial Database. Amazon, flipkart, olx, google etc.

Database management System (DBMS)

A DBMS is a collection of programs that Enables users to create & maintain the database.

It allows to access data from Database & Storing data, Processing queries etc. **Database + DBMS = Database system.**

Why a database?

(or)

Purpose of database.

Databases Can store very large number of records Efficiently. (they take up little Space).

- It is very quick and easy to find information
- It is easy to add new data & to Edit (or) delete old data.
- Data Can be Searched easily.
- Data Can be sorted easily.

Ex: based on register-num (index) either in Ascending (or) decending order...

- Data first registered order.
- Data Can be imported into other applications

Ex: mail-merge.

- More than one person Can access the Same database at the Same time = multi access
- Security may be better than in paper files.
- Duplication of data is not allowed.
- Providing Backup & Recovery!
- Providing multiuser Interfaces.

Ex: includes menu-driven interface, Natural Language, query language etc.

Key features of DBMS

- **Data modeling**: Tools to create and modify data models, defining the structure and relationships within the database.
- **Data storage and retrieval**: Efficient mechanisms for storing data and executing queries to retrieve it quickly.
- **Concurrency control**: Ensures multiple users can access the database simultaneously without conflicts.
- **Data integrity and security**: Enforces rules to maintain accurate and secure data, including access controls and encryption.
- **Backup and recovery**: Protects data with regular backups and enables recovery in case of system failures.

Purpose of database system in DBMS

In the early days of computing (like the 1960s), organizations used file-based systems to manage data. For example, a university might store student, instructor, and course information in separate files. Programs were written for each specific task—like adding new students or generating transcripts. Over time, managing these files became complicated due to several issues:

- **Data redundancy and inconsistency**: the same data might be stored in multiple places, leading to inconsistency. For example, a student's address might be updated in one file but not another.
- **Difficulty in accessing data:** new queries required writing new programs, making data retrieval slow and inflexible.
- **Data isolation:** data stored in different formats across multiple files made it hard to retrieve and combine information.
- **Integrity problems**: enforcing rules (like preventing negative account balances) was difficult, especially when rules changed.
- **Atomicity issues**: if a system fails during a task (like transferring money), partial updates could leave data in an inconsistent state.
- **Concurrent-access anomalies**: when multiple users update data at the same time, inconsistencies can occur without proper control.
- To overcome these limitations, **database management systems (DBMS)** were introduced. A DBMS provides a **centralized, structured, and secure way** to store, manage, and retrieve data efficiently, ensuring **data integrity, consistency, and ease of access**.

Classification of Database management system

Database management systems can be classified based on several factors such as **data model**, **user support**, **distribution**, and **usage**. Below is a comprehensive breakdown:

1. Based on data model

- The **data model** defines how data is logically structured and manipulated.

A. Hierarchical DBMS

- Structure: data is organized in a **tree-like** structure with parent-child relationships.
- Example: a university system might have a hierarchy like **university → departments → courses → students.**

- Limitation: a child can have only one parent.

B. Network DBMS

- Structure: more flexible than hierarchical; supports **many-to-many relationships** using graph structures.
- Advantage: a child can have multiple parents.
- Examples: integrated data store (ids), idms, **Airline reservations – flight → passengers ↔ bookings**

C. Relational DBMS (RDBMS)

- Structure: data is stored in **tables (rows and columns).**
- Key features: use of sql for data management, normalization, constraints, and acid properties.
- Examples: mysql, oracle, sql server, postgresql, **University system** – students, courses, results.

D. Object-oriented DBMS (OODBMS)

- Structure: data is stored as **objects**, similar to object-oriented programming.
- Features: supports complex data types like images, audio, video, etc.
- Examples: objectdb, db4o, **Multimedia apps** – store videos, audio, images with metadata

E. Document-oriented DBMS (nosql)

- Structure: data is stored in **documents** (e.G., JSON, XML), not in rows/columns.
- Usage: ideal for web apps, cms, real-time big data applications.
- Examples: mongodb, couchdb, **E-commerce platforms** – products stored as JSON documents

F. Key-value store DBMS

- Structure: each item is stored as a **key-value pair**.
- Advantage: fast data retrieval, suitable for caching.
- Examples: redis, amazon dynamodb, **Online shopping cart or session storage**

G. Column-oriented DBMS

- Structure: data is stored column-wise instead of row-wise.
- Usage: best for olap (analytical queries), data warehousing.
- Examples: apache hbase, apache Cassandra, Business analytics, reporting tools.

H. Graph DBMS

- Structure: uses nodes and edges to represent data and relationships.
- Usage: social networks, recommendation engines.
- Examples: neo4j, amazon Neptune, **Social networks** – users, friends, followers, likes.

2. Based On Number of Users

A. Single-user DBMS

- Description: only one user can access the system at a time.
- Usage: typically used for personal or small desktop applications.
- Example: microsoft access, **Personal budget tracking** or **small office records**

B. Multi-user DBMS

- Description: allows multiple users to access the database concurrently.
- Usage: enterprise systems, e-commerce, banking, etc.
- Examples: mysql, oracle, postgresql, **Banking, online shopping, college ERP systems**

3. Based On Database Location (Distribution)

A. Centralized DBMS

- Description: all data is stored and maintained on a single system/server.
- Limitation: single point of failure; scalability is limited.
- Examples: ibm db2 on mainframes, **University admin system**

B. Distributed DBMS

- Description: data is distributed across multiple locations or systems, yet appears as a single database to users.
- Advantage: high availability, faster access in regional systems.
- Examples: apache cassandra, google spanner, **Bank branches across cities**, **online multiplayer games.**

C. Cloud-based DBMS

- Description: hosted on cloud platforms and accessible via the internet.
- Advantage: scalable, cost-efficient, easy to maintain.
- Examples: amazon rds, google firebase, microsoft azure sql, **E-commerce platforms**, **mobile apps**, **IoT systems.**

4. Based on access type / use case

A. OLTP (online transaction processing)

- Purpose: handles real-time transactions like inserting, updating, and deleting records.
- Features: fast response, data integrity, short and frequent queries.
- Examples: oracle, mysql, sql server, **ATM transactions**, **shopping cart updates**, **airline booking**

B. OLAP (online analytical processing)

- Purpose: handles complex queries used for analytics and business intelligence.
- Features: read-intensive, supports multi-dimensional analysis (data cubes).
- Examples: sap hana, amazon redshift, snowflake, **Sales forecasting**, **customer behavior analysis.**

5. Based on the purpose

They can be classified as specific and general purpose

- **Specific or special purpose:** built for specific application

  Example: ATM machine

- **General purpose:** built for many applications

  Example: personal computers.

6. Based on type of software

They can be classified as homogeneous and heterogeneous DBMS

- **Homogeneous:** contains same DBMS software at multiple sites

  Example: **Telecom companies (microsoft SQL server)**

  Telecom providers like **airtel** or **vodafone** may use **microsoft SQL server** in all regional offices. Same DBMS helps manage user data, call records, billing data, etc.

- **Heterogeneous:** different software at multiple sites

Example: **Healthcare systems**

Hospitals and labs may use:

- **Mongodb** to store unstructured medical images and reports,
- **Mysql/postgresql** for patient records,
- **Oracle** for financial systems.

All systems are connected using **healthcare integration platforms** like hl7, mirth connect.

Applications of Database :-

1. **Telecom:**

   There is a database to keeps track of information regarding Calls made, Network usage, Customer details etc

2. **Industry:**

   Data about manufacturing unit, warehouse (or) distribution Centre is stored in database.

3. **Banking System:**

   For storing Customer info, tracking day to day Credit & debit transactions, generating bank statements etc

4. **Sales:**

   To store Customer information, production information & invoice details.

5. **Airlines:**

   Reservation information along with flight Schedule is stored in database.

6. **Education Sector:**
   Database system are frequently used in Schools & Colleges to store & retrieve the data regarding student details, Staff details, Course details, Exam details etc.
7. **Online shopping:** The online shopping websites such. Amazon flipkart etc. These sites store the product information, your addresses & preferences, Credit details etc.

DBMS Features:

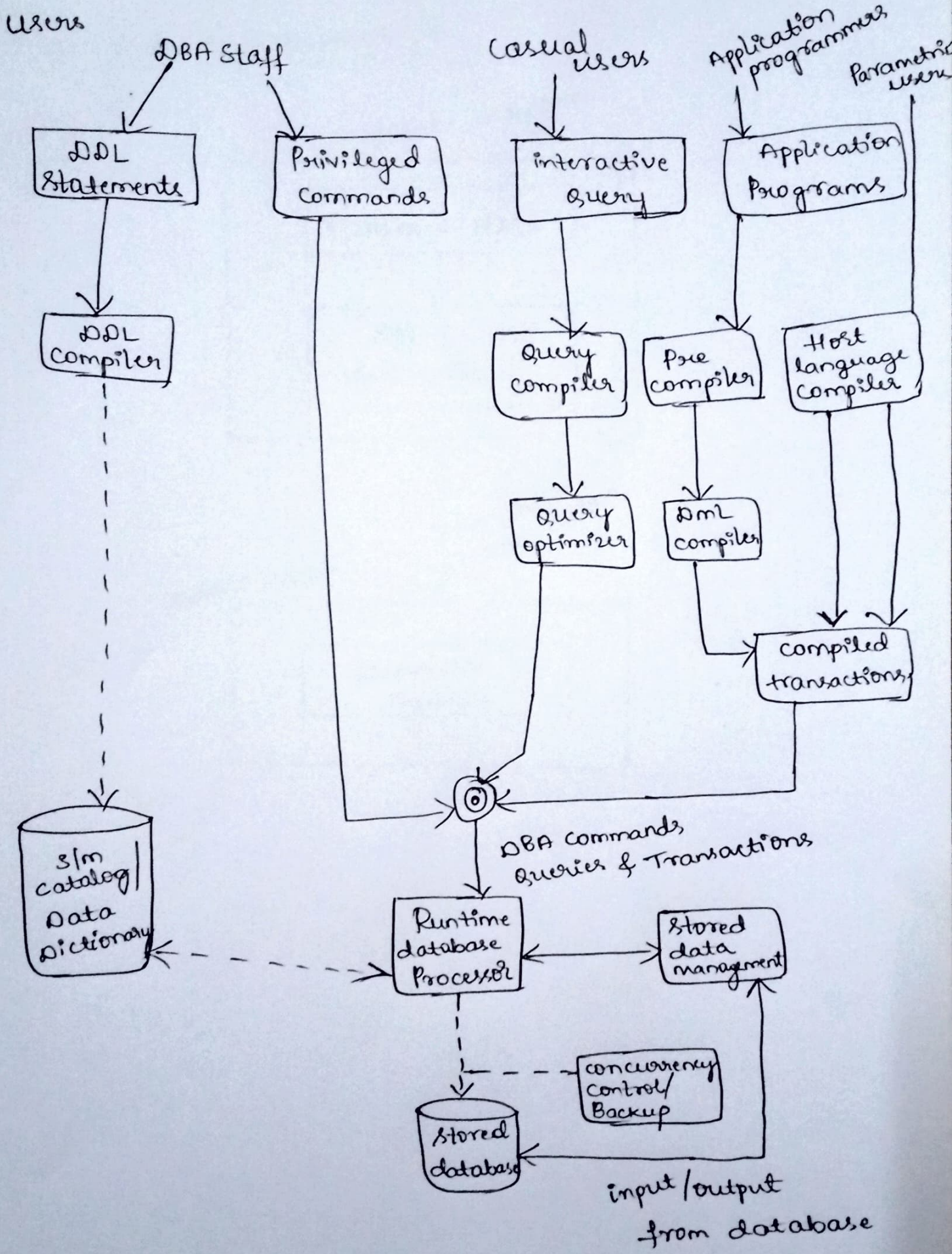
1. Minimum Duplication: All the information in database management System occurs only once. So Chances of duplicity are very less.
2. Provides high level of security: Only the authorized persons are allowed to access the data from the database.
3. Permanent storage of Data: DBMS stores all the data files Permanently & there is no Loss of data. Chance of any loss of data.
4. Multi-user Access: Multiple users can access all Kind of data & information stored in one data Store
5. Providing Backup & Recovery: A DBMS provides, facilities for recovering from hardware (or) Software failures.
6. Providing multiple user Interface: DBMS provides a variety of User interfaces such as menu-driven interface, Form based Natural language etc.

Functional Components of DBMS:

The Component modules of Dems is divided into two parts.

1. The top half refers do the Various users of the database & their interfaces.
2. The lower half refers to the storage of data & processing of transactions.
3. Top half Contains interfaces for the DBA staff, Casual users, application programmers & parametric users.
4. The DDL Compiler processes the DDL statements & stores the description of Schema in the DBMS Catalog.
5. Query Compiler accepts interactive queries & Converts them into intermediate form & passes it onto the query optimizer.
6. Query optimizer would further optimize queries by Elimination of redundancies, reordering of operations etc.
7. It can Consult Catalog for more information & then makes a call to runtime Database Processor.
8. Pre compiler Extracts DML Commands from an application program & sends it do DML Compiler. The rest of the Sent to the host languages.
9. The object code for DML Commands & rest of the program are linked by forming Canned transactions (Compiled).

10. In lower half - The runtime databases processor Executes privilaged Commands, queries & transactions.
11. The Data manager helps the run-time Database & also uses basic operating system Services for Carrying out input-output operations from DB.


Users
DBA Staff
Casual users
Application programmers
Parametric users
DDL Statements
Privileged Commands
interactive Query
Application Programs
DDL Compiler
Query compiler
Pre compiler
Host language compiler
Query optimizer
DML compiler
compiled transactions
DBA commands
Queries & Transactions
s/m catalog / Data Dictionary
Runtime database Processor
Stored data management
concurrency control / Backup
Stored database
input / output from database

Types of DBMS architecture

There Are three types of DBMS architecture

(1) Single tier architecture
(2) Two tier architecture
(3) Three tier architecture

Single tier architecture :

In this type of architecture, the database readily available on the client machine, any request made by client doesn't require a Network Connection to perform the action on the database.

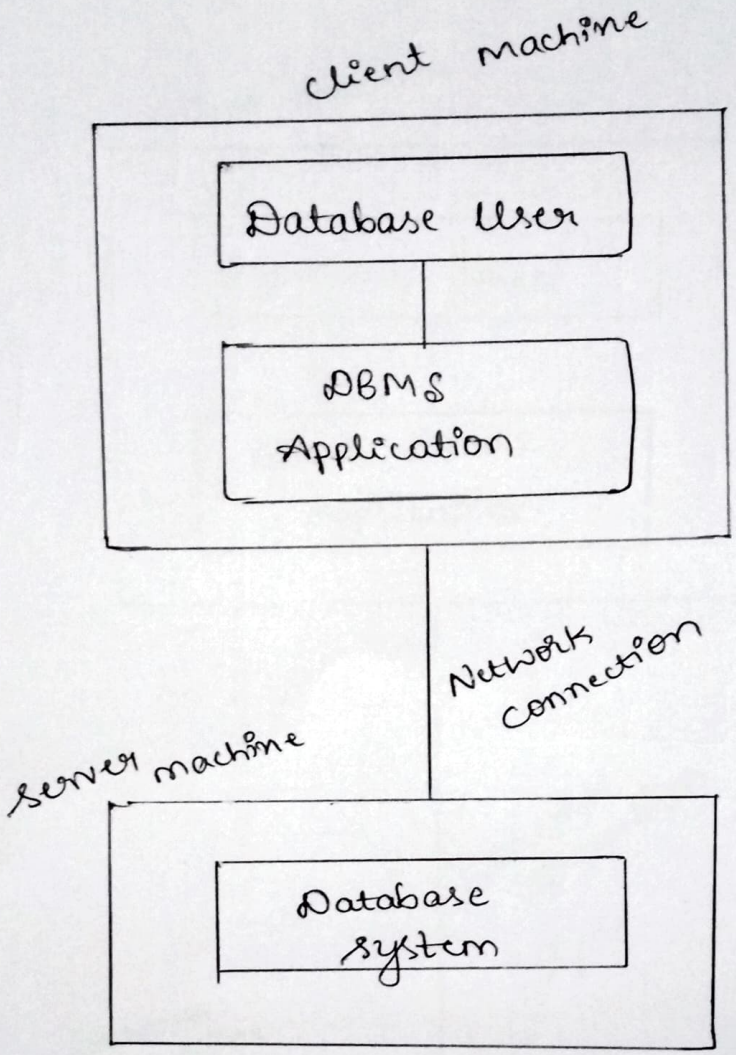
Ex: if we want to fetch the records of Employee from the database & the database is available on your Computer System.

- So the request to fetch Employee details will be done by your Computer Database itself.
- This type of system is generally referred as Local database system.

Two-tier architecture:

- Server: is a powerful system Containing both hardware & Software that provides Sevices to the clients.
- Client: provides user interfaces and local processing power to run the application.
- In two-tier architecture, the Database system is present at server machine and the DBMS application is present at the client machine.
- Both client & Server are Connected with Network shown in the diagram.
- client machine makes a request to access the database present at Server using query language like SQL.
- The Server perform the request on the Perform database and returns the result back to the Client.

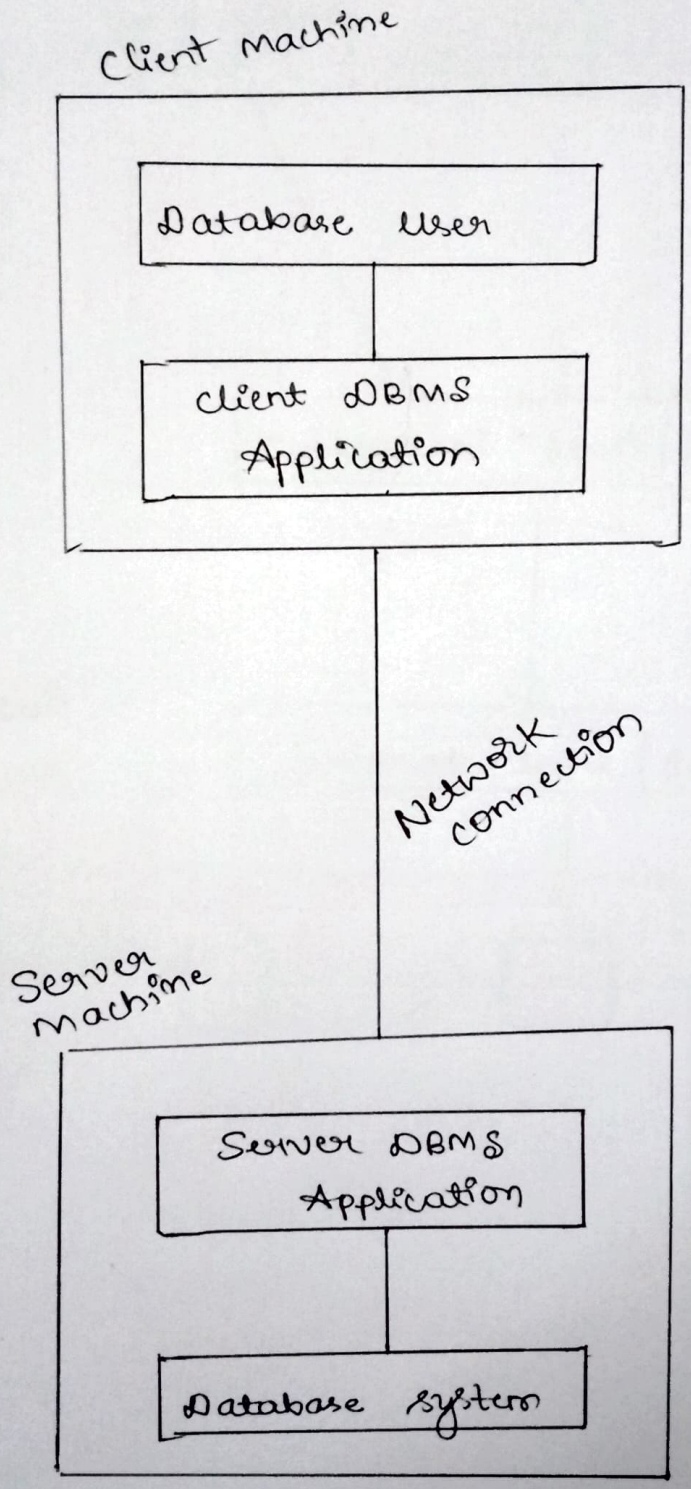
Two-tier Architecture



Three-tier architecture:

- In three-tier architecture, another layer is Present between the client & Server machines.
- In this architecture, the client application doesn't Communicate directly with the database systems present at the Server.
- The client application Communicates with Server application & Server application internally Communicates with the database system present at the Server.

Three - tier Architecture



View of Data in DBMS

View of data in DBMS tells how the data is Visualized at each level.

1. **Data abstraction:**
   - Allow developers to Keep Complex data structures away from the users.
   - The developers achieve this by hiding the data through levels of abstractions.
   - To achieve data abstraction, we discuss a Three- Schema architecture
   - The main objective of the three-Schema architecture is to separate the user application & physical storage database.

The three-Schema architecture defines the View of data at three levels.

(1) Physical level (internal level)

(2) Logical level (Conceptual level)

(3) View level (External level)

Physical level:

- Describes how the data is Stored in the hardware.
- It also describes how the data can be accessed.
- It is the lowest level of data abstraction.
- Only the database administrator operates at this level.

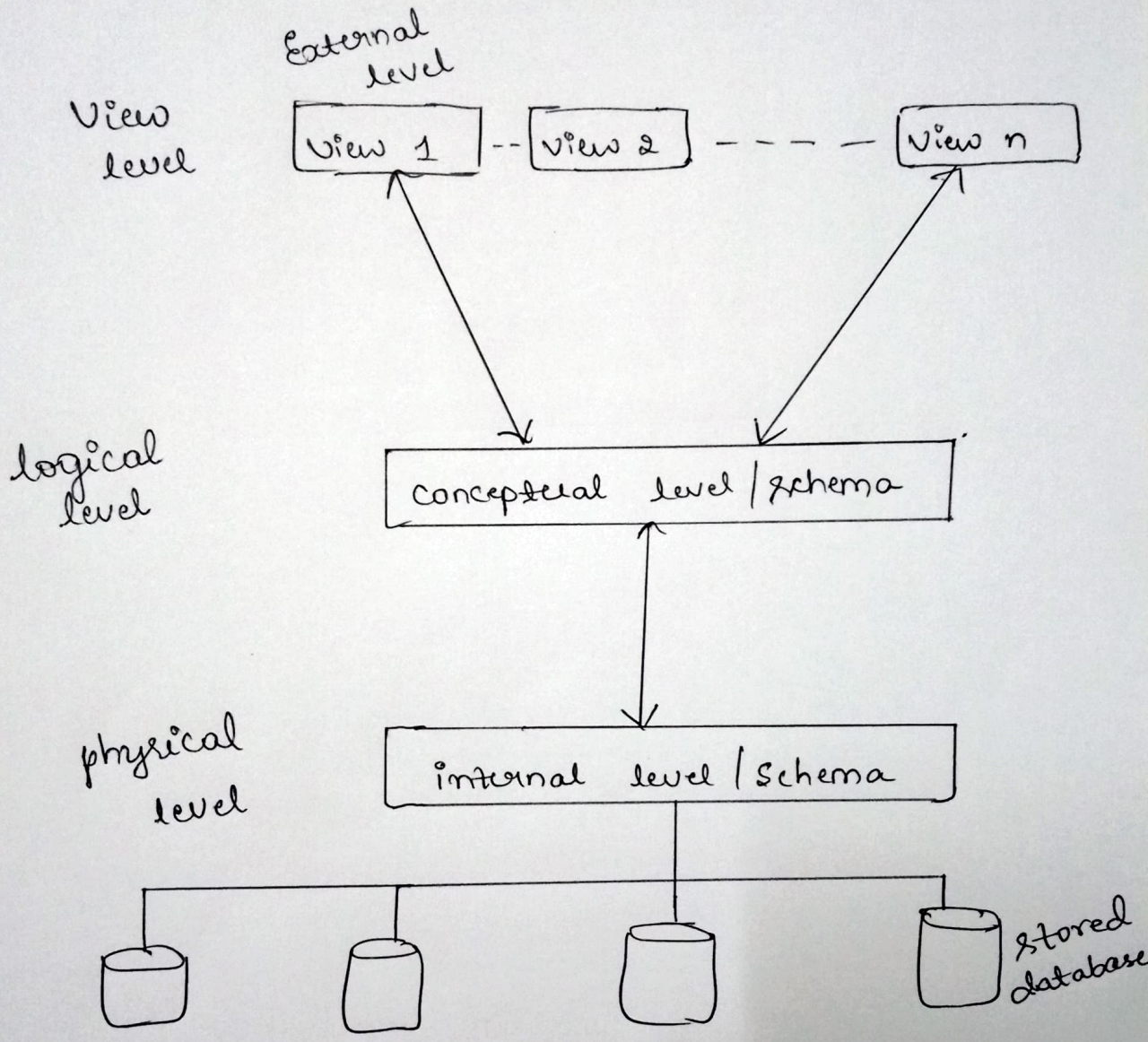
Logical level / Conceptual level:

- The data is stored in the form of Entities, their data types, the relationship among Entities.
- It also Specifies the user operations to retrieve (or) modify the data & Certain Constraints On the data to provide Security.
- The developer and database administrator can operates at this level.

View level /user level External level

- It is the highest level of data abstraction
- It displays data in which the user is interested.
- View level can describe many views of the Same data, the user retrieves the information using different applications.

View of data at 3 levels



2. Data Independence:

Data independence as the Capacity to change the data at one level without having the change at the next higher level.

Two types of data independence:

1. Logical data independence

it refers to the Capacity to change Conceptual schema without having to change the External Schema.

Ex. We may change the Conceptual Schema to expand the database (by adding a attribute (or) data item) without affecting the External Schema.

2. Physical data independence

It refers to the Capacity to change the internal Schema without having to change the Conceptual Schema.

Ex-Re-organizing the file in the internal schema without changes in the Conceptual Schema.

Database Users:

Role & Responsibilities of DBA:.

1. DBA (Database Administrator)

- Database Administrators responsible for administering Database Environment.
- DBA is responsible for authorizing access to the database by Creating an account id & password for the user to Access the database.
- DBA is also responsible for providing Security to the data.
- DBA also monitors the recovery and backup & provide technical support.
- DBA repairs damage Caused due to hardware and / or Software failures.
- The DBA has a DBA account in the DBMS which Called a Superuser account or System account.
- Responsible for performance issues.
- Plans the Storage Structure & access Strategy.

2. End Users

The people access to the database for querying, updating & generating reports.

1. Casual users:

Casual users are the users who occasionally use/access the database, but they may require different information each time from the database.

Ex: Students.

2. Naive/Parametric End users:

Parametric End users are the users who don't have any DBMS knowledge but they frequently use the database application in their daily life.

Ex: Clerks in bank, Bank tellers Check Account balances, withdrawals, deposits etc

3. Sophisticated users:

End users who thoroughly familiarize themselves with the facilities of the DBMS in order to the implement their applications. Ex: Engineers, Scientists, business analyst etc.

4. Standalone End users

Maintain personal databases by using read-made software packages that provide easy to- use.

Ex: Tally, Scientists etc

3. Database Designers:

Database Designers are the Users who design the structure of database which includes tables, Constraints, triggers, Stored procedures.

He/She Controls what data must be stored & how the data items to be related.

4. System Analyst & Application Programmers

- System Analysts identify the requirements of naive / parametric users & develop Specifications for Canned transactions.
- Application programmers implement these Specifications as programs, test them, debug documentation etc
- System Analyst & Application programmers together Called Software Engineer.

Example of database application: Recruitment database

A recruiting database is a tool used for storing all information on jobs and candidates matching applicants with Jobs, and streamlining the Entire recruitment process.

It includes

1. Resume Management
   - Supporting for multi-format resume
   - Resume uploading & downloading
   - Resume Screening & matching with Jobs openings
2. Recruiting information management
   - Creating a talent pool of applicants,
   - Viewing & Editing Candidate records
   - Possibility to attach the Certificates etc

3. Security & compliance

- Candidate data Encryption.
- Multi-factor Authentication